Nombre: José Angel Martínez Herrera	**Matrícula**: 2147975
Nombre: Fernanda Lozano Ovalle	**Matrícula**: 2147342
Nombre: Mauricio José Marcos Mesías	**Matrícula**: 2147654
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Diseño Arquitectural del Sistema FitZone
Fecha: 14/10/2026	
Bibliografia (Formato APA)	

Objetivo:

Este documento presenta el diseño arquitectural del sistema de FitZone, a partir del análisis de requerimientos de la Unidad 2 (BRD, historias de usuario, BPMN y glosario). Su propósito es organizar las reglas y responsabilidades del proceso de membresías, accesos, reservación de clases y listas de espera, separando dominio, aplicación e infraestructura. Cada decisión de diseño se justifica con evidencia del caso y se registra en ADR, con trazabilidad hacia las necesidades, reglas e historias de usuario.

Diseño Arquitectural del Sistema FitZone

Link de GitHub

https://github.com/fernandalozano06/Actividad-Fundamental-3.git

Alcance

Reglas que gobiernan el diseño

Solo el administrador puede cancelar, reprogramar o reasignar clases; los instructores únicamente reportan su indisponibilidad (BR-01 y BR-02). Recepción puede gestionar reservas y cancelaciones, pero no modificar membresías (BR-03 y BR-04). La capacidad de las clases no debe superarse (BR-05), y las membresías vencidas o los planes incompatibles impiden el acceso o la reserva (BR-06 y BR-07). Los no-shows se registran sin aplicar consecuencias (BR-08 y BR-09), mientras que la política de cancelación permanece pendiente de definición (BR-10).

Dentro del alcance

El sistema permitirá consultar clases, gestionar reservas y listas de espera, validar membresías y accesos, registrar asistencias y no-shows, administrar cambios de clases y notificar a los socios. También incluirá reportes de indisponibilidad de instructores e indicadores de ocupación, no-shows y listas de espera.

Fuera del alcance

Se excluyen los pagos, servicios adicionales, contratación o modificación de membresías, nómina, contabilidad y gestión completa de entrenamientos personalizados. FitZone deberá definir las políticas de cancelación, las consecuencias de los no-shows y el tiempo límite de confirmación antes de la implementación definitiva.

1. Preparación del caso (Etapa 1)

1.1 Proceso principal y casos de uso

Proceso principal: **reservación de clases, control de acceso y lista de espera** (Necesidades N-01 a N-05; prioridades alta y media-alta del BRD). Se eligieron cinco casos de uso: un flujo normal (CU-01), flujos con rechazo o excepción (CU-01 alterno, CU-02, CU-04) y flujos de estado (CU-03, CU-05).

ID	Caso de uso	Actor	Reglas	Resultado esperado	Trazabilidad
CU-01	Reservar clase (flujo normal; alterno: membresía no vigente o clase fuera del plan)	Socio / Recepción	BR-03, BR-05, BR-06, BR-07	Reserva activa asociada al socio y la clase con confirmación; o rechazo con motivo y sugerencia de actualizar plan	US-02, US-03, US-04; BPMN-1
CU-02	Unirse a la lista de espera (clase llena)	Socio	BR-05, BR-06, BR-07	Socio al final de la lista con su posición	US-05, US-16; BPMN-1 («Lista de espera»)
CU-03	Cancelar reserva y ofrecer el lugar al primero de la lista (con confirmación o vencimiento de la oferta)	Socio / Recepción	BR-03, BR-05, BR-10	Lugar liberado y ofrecido; si no se confirma, pasa al siguiente	US-07, US-06; BPMN-1 (evento «Cancela su reserva»)
CU-04	Validar acceso y registrar asistencia	Recepción	BR-06, BR-07	Acceso válido o denegado; asistencia registrada	US-08, US-19; BPMN-2
CU-05	Registrar no-show al cerrar la clase	Sistema	BR-08, BR-09	Reservas sin asistencia marcadas como no-show e historial actualizado, sin sanción	US-10; BPMN-2

Servicios secundarios (con firma pero sin diagrama detallado): administrar clase (US-12, US-13; BR-01, BR-05), reportar indisponibilidad del instructor (US-14; BR-02), notificar cambios (US-15) y consultas (US-01, US-11, US-16, US-17, US-18).

2. Elección del patrón de lógica de negocio (Etapa 2)

2.1 Tabla comparativa con criterios del caso

Criterio del caso	Transaction Script	Table Module	**Domain Model**	Evidencia U2
Complejidad de reglas	Un procedimiento por caso de uso. BR-05, BR-06 y BR-07 se repiten en reservar, reservar por Recepción, confirmar desde la lista y registrar asistencia.	Un módulo por tabla (Reservas, Clases, Membresías). Ningún módulo es dueño de la regla que combina vigencia, plan y cupo.	Cada regla vive en el objeto que la posee: cupo en `ClaseGrupal`, vigencia en `Membresia`, inclusión en `PlanDeMembresia`. Un Domain Service las combina.	BR-05, BR-06, BR-07; BPMN-1 con tres gateways en cascada
Cambios esperados	Cada nueva política (BR-09, BR-10, tiempo de confirmación) añade condicionales a varios scripts.	El cambio se reparte entre varios módulos.	Una política es una estrategia sustituible, con un solo punto de cambio.	BR-09, BR-10; Restricciones y Supuestos del BRD
Interacción entre conceptos	Un script consulta y modifica Socio, Membresía, Plan, Clase, Reserva y Lista, con alto acoplamiento a tablas.	La coordinación entre módulos queda fuera de ellos.	Grafo de objetos con límites de consistencia (Aggregates) y eventos.	US-02, US-05, US-06, US-19
Estados e invariantes	El estado son columnas y las transiciones se validan a mano, por lo que es fácil violar el cupo o el orden de la lista.	Igual que Transaction Script, con estado en filas.	Transiciones encapsuladas: la reserva solo cambia de estado mediante métodos del Aggregate que protegen las invariantes.	BR-05, BR-08; US-06 (lista con tiempo de confirmación); US-10

Criterio del caso	Transaction Script	Table Module	**Domain Model**	Evidencia U2
Roles y permisos	Verificación de rol repetida en cada script.	Verificación dispersa.	`Actor` con `Rol` y métodos que exigen el permiso.	BR-01, BR-02, BR-03, BR-04; Supuestos del BRD
Reportes e indicadores (criterio adicional)	Adecuado: consultas directas.	Muy adecuado para datos tabulares.	No es el mejor ajuste para agregaciones masivas.	N-07, US-11, US-18

2.2 Ventajas, riesgos y consecuencias en FitZone

Patrón	Ventajas	Riesgos y costos
Transaction Script	Rápido de construir. Adecuado para US-01, US-11, US-17 y US-18 (solo lectura).	Duplicación de BR-05 a BR-07 en cuatro o más scripts. Cada cambio de política obliga a editar varios. Facilita accesos inconsistentes (problema que N-01 busca eliminar).
Table Module	Buen encaje con reportes tabulares.	Las reglas que cruzan Membresía, Plan y Clase no tienen dueño. Estados en filas sin protección.
Domain Model	Reglas con dueño y con nombres del glosario. Invariantes verificables con pruebas unitarias. Cambios localizados.	Más clases y mapeo objeto-relacional. Curva de aprendizaje. Cuidado con la concurrencia sobre `ClaseGrupal`. Posible sobreingeniería en consultas.

2.3 Decisión

Patrón principal: Domain Model, con una combinación delimitada: las **consultas** (calendario, posición en la lista, reservas del socio, indicadores) usan lectura directa estilo Transaction Script/Table Module en el lado de consulta de CQRS. Las **operaciones que cambian estado** pasan siempre por el dominio.

Evidencias de la Unidad 2 que sustentan la decisión:

1. **BR-05, BR-06 y BR-07 con los gateways de BPMN-1.** Una reserva depende, en orden, de membresía vigente, clase incluida en el plan y cupo disponible, y esas tres reglas aparecen otra vez en la lista de espera (US-05) y en el acceso (US-08). Con Transaction Script esa cadena se copiaría en cada caso de uso.

2. **US-06, US-10 y BR-08: ciclos de vida con tiempo.** La reserva pasa por activa, cancelada, asistida o no-show. La entrada en lista pasa por esperando, ofertada, confirmada o expirada. Requieren transiciones protegidas, no columnas editables.

3. **BR-09, BR-10 y las Restricciones del BRD: políticas que cambiarán.** Todavía no existen la consecuencia del no-show, la política de cancelación ni el tiempo de confirmación. El dominio debe poder recibirlas sin reescribir los casos de uso.

4. **BR-01 a BR-04 y los Supuestos:** las acciones administrativas deben diferenciarse de las de Recepción e Instructor, y esos permisos son reglas del negocio, no detalles de interfaz.

Consecuencia negativa aceptada: mayor costo inicial de modelado y mapeo, y la necesidad de control de concurrencia sobre `ClaseGrupal` (último cupo). Se acepta porque el problema principal de FitZone (no-shows, lugares vacíos y lista de espera manual) es justamente de reglas, estados y consistencia.

3. Mapa de dominio (Etapa 3)

3.1 Lenguaje ubicuo

Término del glosario	Significado	Representación en el modelo
Socio	Cliente con una membresía	`Socio` (entidad de referencia), `SocioId`
Membresía	Suscripción que da acceso según el plan	`Membresia` (raíz)
Plan	Nivel que define qué clases están incluidas	`PlanDeMembresia` (raíz)
Membresía vigente / Vencimiento	Dentro de su periodo de validez / fecha en que termina	`Membresia.estaVigenteEn(instante)`, `VigenciaDeMembresia`
Clase grupal	Sesión con instructor, horario y capacidad máxima	`ClaseGrupal` (raíz)
Calendario de clases	Clases programadas con horarios y disponibilidad	Modelo de lectura `CalendarioDeClasesVm`
Cupo o capacidad / Lugar disponible	Máximo de socios / espacio libre dentro de la capacidad	`Capacidad`, `ClaseGrupal.lugaresDisponibles()`
Reserva	Registro que aparta el lugar de un socio en una clase	`Reserva` (entidad dentro de `ClaseGrupal`)
Lista de espera	Socios que piden lugar en una clase llena	`ListaDeEspera`, `EntradaDeListaDeEspera`
Cancelación de reserva	El socio libera voluntariamente su lugar	`ClaseGrupal.cancelarReserva(...)`, evento `ReservaCancelada`
Cancelación de clase	Suspensión por decisión administrativa	`ClaseGrupal.cancelar(actor)`, evento `ClaseCancelada`
Reprogramación	Cambio de fecha u horario	`ClaseGrupal.reprogramar(horario, actor)`
No-show	Reservó, no canceló y no se presentó	`EstadoDeReserva.NoShow`, evento `NoShowRegistrado`
Asistencia / Historial de asistencia	Confirma que tomó la clase / asistencias, cancelaciones y no-shows del socio	`EstadoDeReserva.Asistida`, `HistorialDeAsistencia`

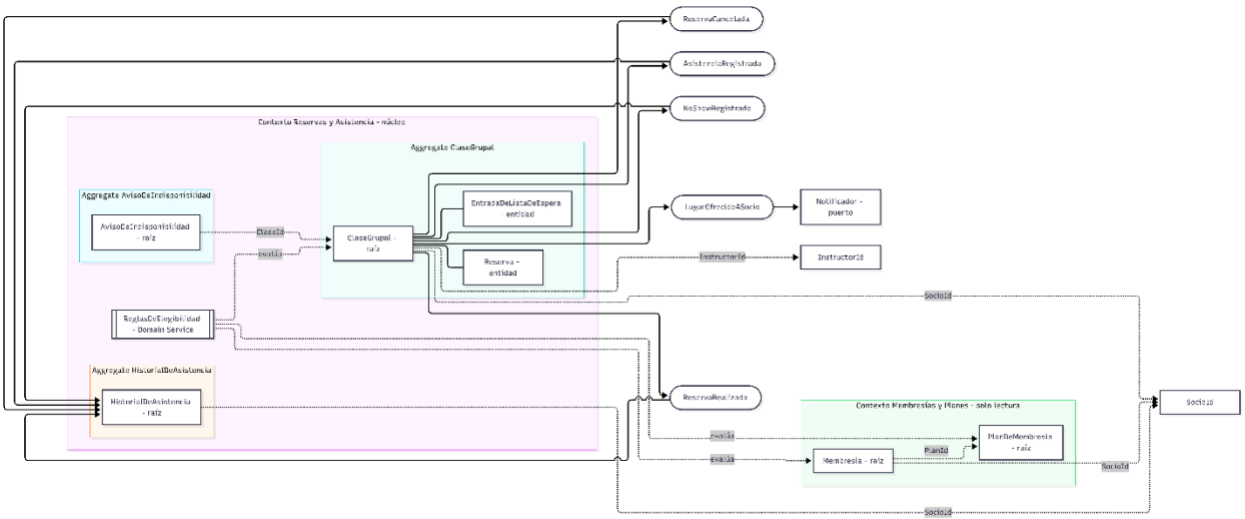
Término del glosario	Significado	Representación en el modelo
Acceso	Autorización de entrada según membresía y reserva	`ResultadoDeAcceso` (Value Object)
Recepción, Instructor, Administrador	Roles del negocio	`Rol` dentro de `Actor`
Servicio adicional, Entrenamiento personal, Adeudo, Suscripción	Conceptos fuera del alcance del diseño detallado	No modelados (H-05)

Términos nuevos propuestos para actualizar el glosario:

Término	Definición	Origen
Oferta de lugar	Lugar liberado que se ofrece al primer socio de la lista y que debe confirmar en su ventana	US-06
Ventana de confirmación	Tiempo máximo para confirmar una oferta (valor pendiente, SP-02)	US-06
Posición en la lista	Orden de llegada del socio en la lista de espera	US-05, US-16
Aviso de indisponibilidad	Registro de que el instructor no podrá impartir una clase	BR-02, US-14
Política de cancelación	Condiciones y tiempos para cancelar una reserva (pendiente, SP-01)	BR-10, US-07
Elegibilidad para reservar	Resultado de evaluar membresía vigente y clase incluida en el plan	BR-06, BR-07
Actor	Quien ejecuta una acción, con su rol (Socio, Recepción, Instructor, Administrador, Sistema)	BR-01 a BR-04

3.2 Límites (contextos) y diagrama del mapa de dominio

- **Reservas y Asistencia (núcleo, este diseño):** clases, reservas, lista de espera, asistencia, no-show, historial, avisos.
- **Membresías y Planes (aguas arriba, solo lectura):** FitZone ya vende y administra membresías; contratar o modificarlas está fuera de alcance (BRD, BR-04). Este contexto consume `Membresia` y `PlanDeMembresia` mediante repositorios de lectura.
- **Notificaciones (soporte):** puerto `Notificador`, sin modelo propio (SP-07).



3.3 Fichas de Aggregates

Aggregate 1: `ClaseGrupal` (raíz). Contiene `Reserva` y `EntradaDeListaDeEspera`.

Elemento	Detalle
Responsabilidades	Controlar capacidad, reservar, cancelar reserva, administrar la lista de espera (alta, oferta, confirmación, expiración), registrar asistencia y no-shows, cancelar, reprogramar, reasignar y configurar capacidad.
Invariantes	**INV-1** reservas activas más ofertas vigentes no superan la capacidad (BR-05, N-05). **INV-2** solo se reserva en una clase programada. **INV-3** la lista de espera solo admite socios cuando la clase está llena y mantiene orden de llegada; la oferta va al primero (US-05, US-06). **INV-4** un socio no tiene dos reservas activas ni dos entradas vigentes en la misma clase (SP-08). **INV-5** un no-show es una reserva activa, sin cancelar y sin asistencia al terminar el horario (BR-08, US-10). **INV-6** solo el Administrador cancela, reprograma, reasigna o fija capacidad (BR-01, BR-05). **INV-7** Recepción puede reservar y cancelar en nombre del socio (BR-03). **INV-8** capacidad mayor que cero.
Eventos	`ReservaRealizada`, `ReservaCancelada`, `SocioAgregadoAListaDeEspera`, `LugarLiberado`, `LugarOfrecidoASocio`, `LugarConfirmado`, `OfertaDeLugarExpirada`, `AsistenciaRegistrada`, `NoShowRegistrado`, `ClaseCancelada`, `ClaseReprogramada`, `ClaseReasignada`, `CapacidadDeClaseConfigurada`

Aggregate 2: `Membresia` (raíz) y `PlanDeMembresia` (raíz). Invariantes: vigente si el instante está entre inicio y vencimiento y no está suspendida (BR-06); el plan define los tipos de clase incluidos (BR-07). Recepción no puede modificarla (BR-04): no existe caso de uso de modificación en este sistema y cualquier mutador exige un permiso que Recepción no tiene. No emite eventos en este diseño.

Aggregate 3: `HistorialDeAsistencia` (raíz, uno por socio). Registra asistencias, cancelaciones y no-shows (glosario, US-10, US-19) y calcula el número de no-shows por socio (indicador 2 del BRD). Se actualiza reaccionando a eventos, en otra transacción.

Aggregate 4: `AvisoDeIndisponibilidad` (raíz). Identidad y ciclo de vida propios: pendiente, atendido (BR-02, US-14). Evento: `IndisponibilidadDeInstructorReportada`. El Instructor no puede cancelar ni reprogramar (BR-01, BR-02).

3.4 Value Objects, Domain Service y eventos

Value Object	Regla que protege
`ClaseId`, `SocioId`, `ReservaId`, `MembresiaId`, `PlanId`, `InstructorId`	Identificadores tipados, sin mezclar entre conceptos
`Horario` (inicio, fin)	El inicio es anterior al fin
`Capacidad`	Entero mayor que cero (BR-05)
`VigenciaDeMembresia` (inicio, vencimiento)	Consulta de vigencia por fecha (BR-06)
`PosicionEnLista`	Orden de llegada (US-05, US-16)
`VentanaDeConfirmacion`	Duración configurable (SP-02)
`Actor` (id y `Rol`)	Permisos por rol (BR-01 a BR-04)
`ResultadoDeElegibilidad`	Solo lo crea `ReglasDeElegibilidad`; contiene `Elegible` o `Rechazada` con motivo `MEMBRESIA_NO_VIGENTE` o `CLASE_FUERA_DEL_PLAN`
`ResultadoDeAcceso`	Acceso válido o denegado con motivo (US-08)
`RegistroDeHistorial`	Tipo (asistencia, cancelación, no-show), clase e instante

Domain Service `ReglasDeElegibilidad`: evalúa membresía vigente (BR-06) y clase incluida en el plan (BR-07) usando `Membresia`, `PlanDeMembresia` y `ClaseGrupal`. No es método de una sola entidad porque cruza tres conceptos. **`PoliticaDeCancelacion`** es una interfaz de dominio (SP-01).

Domain Event (pasado)	Disparador	Reacción
`ReservaRealizada`	Reserva registrada	Historial opcional; indicadores
`ReservaCancelada`	Cancelación voluntaria	`HistorialDeAsistencia`; el mismo Aggregate ofrece el lugar
`LugarOfrecidoASocio`	Oferta al primero de la lista	`Notificador` (US-06)
`OfertaDeLugarExpirada`	Venció la ventana sin confirmar	Notificación opcional; el Aggregate ofrece al siguiente

Domain Event (pasado)	Disparador	Reacción
`AsistenciaRegistrada` / `NoShowRegistrado`	Recepción registra asistencia / cierre de clase	`HistorialDeAsistencia` (BR-08, BR-09)
`ClaseCancelada` / `ClaseReprogramada`	Acción del Administrador	`Notificador` a socios con reserva (US-15)
`IndisponibilidadDeInstructorReportada`	Instructor informa	`Notificador` al Administrador (US-14)

3.5 Decisiones dudosas o descartadas

- **`Reserva` como Aggregate propio:** descartado. El cupo exige consistencia con las demás reservas y con la lista de espera (ADR-002).
- **`ListaDeEspera` como Aggregate propio:** descartado por la misma razón: la oferta consume un lugar de la clase.
- **`Socio` como Aggregate completo:** descartado. En este contexto solo importa su identificador.
- **`Acceso` como entidad:** descartado. En U2 no tiene ciclo de vida propio; es un resultado (`ResultadoDeAcceso`) y la asistencia queda como estado de `Reserva`.
- **`Plan` dentro de `Membresia`:** descartado. Muchos socios comparten un mismo plan.
- **`Notificacion` como entidad:** descartada. Es un efecto, no un concepto con reglas (SP-07).

4. Service Layer (Etapa 4)

Principio: cada método de un Application Service es una intención completa de un actor y sigue siempre la misma secuencia: recibir el Command o Query, abrir la unidad de trabajo, cargar los Aggregates mediante Repositories, pedir al dominio que evalúe o ejecute, guardar, confirmar, publicar eventos y devolver un DTO. El servicio no contiene condiciones de negocio. Lo que el negocio permite, calcula o protege vive en el dominio.

Servicio	Método (caso de uso)	Entrada	Colaboradores	Salida	Reglas que delega (y a quién)
`ServicioDeReservas`	`reservarClase` (CU-01)	`ReservarClaseCommand` (claseId, socioId, actor)	`UnitOfWork`, `ClaseGrupalRepository`, `MembresiaRepository`, `PlanRepository`, `ReglasDeElegibilidad`, `Reloj`	`ReservaDto`	BR-06 y BR-07 a `ReglasDeElegibilidad`. BR-05, INV-2 e INV-4 a `ClaseGrupal.reservar`. BR-03 a `Actor`.
`ServicioDeReservas`	`unirseAListaDeEspera` (CU-02)	`UnirseAListaDeEsperaCommand`	Los mismos	`PosicionDto`	Elegibilidad a `ReglasDeElegibilidad`. «solo si está llena» y el orden a `ClaseGrupal.unirAListaDeEspera` (INV-3).
`ServicioDeReservas`	`cancelarReserva` (CU-03)	`CancelarReservaCommand` (claseId, socioId, actor)	`UnitOfWork`, `ClaseGrupalRepository`, `PoliticaDeCancelacion`, `VentanaDeConfirmacion`, `Reloj`	`void`	BR-10 a `PoliticaDeCancelacion`. BR-03 a `Actor`. La oferta al primero de la lista a `ClaseGrupal.cancelarReserva`.
`ServicioDeReservas`	`confirmarLugarOfrecido` (CU-03)	`ConfirmarLugarOfrecidoCommand`	Como `reservarClase`	`ReservaDto`	Oferta vigente y elegibilidad nueva a `ClaseGrupal.confirmarLugarOfrecido` y `ReglasDeElegibilidad`.

Servicio	Método (caso de uso)	Entrada	Colaboradores	Salida	Reglas que delega (y a quién)
`ServicioDeReservas`	`expirarOfertasVencidas` (CU-03, tarea programada)	Actor `Sistema`	`ClaseGrupalRepository.obtenerClasesConOfertasVencidas`, `UnitOfWork`, `Reloj`	Número de ofertas	Vencimiento y paso al siguiente a `ClaseGrupal.expirarOfertasVencidas`.
`ServicioDeAcceso`	`validarAcceso` (CU-04)	`ValidarAccesoQuery` (socioId, claseId opcional)	Repositories, `ReglasDeElegibilidad`, `Reloj`	`ResultadoDeAccesoDto`	BR-06, BR-07 y reserva activa a `ReglasDeElegibilidad` y `ClaseGrupal.tieneReservaActivaDe`.
`ServicioDeAcceso`	`registrarAsistencia` (CU-04)	`RegistrarAsistenciaCommand` (claseId, socioId, actor)	Los anteriores y `UnitOfWork`	`void`	Vuelve a validar el acceso dentro de la misma transacción y delega a `ClaseGrupal.registrarAsistencia`.
`ServicioDeCierreDeClase`	`registrarNoShows` (CU-05)	`RegistrarNoShowsCommand` (claseId)	`ClaseGrupalRepository`, `UnitOfWork`, `Reloj`	Número de no-shows	BR-08 y BR-09 a `ClaseGrupal.registrarNoShows`.
`ServicioDeAdministracionDeClases`	`configurarCapacidad`, `cancelarClase`, `reprogramarClase`, `reasignarInstructor` (US-12, US-13)	Commands con `actor`	`ClaseGrupalRepository`, `UnitOfWork`	`void`	BR-01 y BR-05 a `ClaseGrupal`, que exige que el actor pueda administrar clases.
`ServicioDeIndisponibilidad`	`reportarIndisponibilidad` (US-14)	`ReportarIndisponibilidadCommand`	`AvisoDeIndisponibilidadRepository`, `UnitOfWork`	`void`	BR-02: solo registra el aviso. No cancela ni reprograma.

Revisión obligatoria de la Service Layer:

Pregunta	Respuesta
¿El nombre del método expresa una acción del negocio?	Sí: `reservarClase`, `cancelarReserva`, `registrarNoShows` usan el glosario.
¿Coordina o decide?	Coordina. Cada condición está en `ClaseGrupal`, `ReglasDeElegibilidad`, `PoliticaDeCancelacion` o `Actor`.
¿Depende de interfaces o de detalles de BD?	De interfaces: Repository, `UnitOfWork`, `Reloj` y notificadores.
¿Inicio y final transaccional claros?	Una transacción por Command: se abre al inicio y termina en `confirmar()` o `revertir()`.
¿Los errores se explican con el negocio?	Sí, ver la lista de errores de dominio.

La autenticación (quién es el usuario) se resuelve en la entrada de infraestructura. La autorización de negocio (qué rol puede cancelar una clase) es regla del dominio (BR-01).

5. Repository y Unit of Work (Etapa 5)

Repository (interfaz de dominio)	Necesidad del dominio	Métodos propuestos	Objeto
`ClaseGrupalRepository`	Recuperar y guardar la clase completa con reservas y lista	`obtenerPorId(ClaseId)`, `guardar(ClaseGrupal)`, `obtenerClasesTerminadasSinCierre(Instante)`, `obtenerClasesConOfertasVencidas(Instante)`	`ClaseGrupal`
`MembresiaRepository`	Conocer la membresía actual del socio (solo lectura)	`obtenerActualDelSocio(SocioId)`	`Membresia`
`PlanRepository`	Conocer qué clases incluye un plan	`obtenerPorId(PlanId)`	`PlanDeMembresia`

Repository (interfaz de dominio)	Necesidad del dominio	Métodos propuestos	Objeto
`HistorialDeAsistenciaRepository`	Acumular asistencias, cancelaciones y no-shows	`obtenerPorSocio(SocioId)`, `guardar(HistorialDeAsistencia)`	`HistorialDeAsistencia`
`AvisoDeIndisponibilidadRepository`	Registrar y atender avisos del instructor	`obtenerPorId(AvisoId)`, `guardar(AvisoDeIndisponibilidad)`	`AvisoDeIndisponibilidad`

Implementación en infraestructura (separada del dominio): repositorios sobre base relacional con ORM. `ClaseGrupal`, `Reserva` y `EntradaDeListaDeEspera` se cargan y guardan juntos. `ClaseGrupal` lleva una columna de versión para concurrencia optimista (dos socios que toman el último lugar). Las consultas SQL no aparecen en las interfaces.

Unit of Work:

Aspecto	Diseño
Inicio	Al entrar al método del Application Service (`iniciar()`).
Contenido	Cambios del Aggregate modificado más sus eventos de dominio, escritos en una tabla de eventos pendientes (bandeja de salida) en la misma transacción.
Regla	Una transacción modifica un solo Aggregate. El historial se actualiza después, por un manejador de eventos con su propia transacción.
Final	`confirmar()` persiste y libera. Cualquier excepción provoca `revertir()`.

Error	Resultado
Rechazo de dominio (`ClaseLlena`, `MembresiaNoVigente`)	Se revierte, no se guarda nada y no se emiten eventos. Se devuelve el motivo del negocio.
Conflicto de versión (último lugar)	Un reintento; si persiste, `ClaseLlena` o conflicto explícito. Nunca se supera la capacidad.
Falla técnica de BD	Se revierte y se informa un error técnico.
Falla al notificar o al actualizar el historial	La reserva ya está confirmada. El evento queda pendiente y se reintenta.

6. CQRS básico (Etapa 6)

Misma base de datos. La separación es de responsabilidades: los Commands cambian estado a través del dominio y las Queries leen modelos propios sin cargar Aggregates.

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificación
Command	`ReservarClaseCommand`	claseId, socioId, actor	`ServicioDeReservas.reservarClase`	`ReservaDto`	Cambia estado y exige BR-05 a BR-07 (US-02, US-04).
Command	`CancelarReservaCommand`	claseId, socioId, actor	`ServicioDeReservas.cancelarReserva`	`void`	Cambia estado, libera y ofrece el lugar (US-07, US-06, BR-10).
Command	`UnirseAListaDeEsperaCommand`, `RegistrarAsistenciaCommand`, `RegistrarNoShowsCommand`	Ver tabla de servicios	Métodos de `ServicioDeReservas`, `ServicioDeAcceso` y `ServicioDeCierreDeClase`	`PosicionDto`, `void`, número	Cambian estado (US-05, US-19, US-10).
Query	`ConsultarCalendarioDeClasesQuery`	rango de fechas, tipo de clase	`ConsultarCalendarioHandler`	`CalendarioDeClasesVm` (clase, tipo, horario, instructor, lugares disponibles, estado)	Lectura para el socio (US-01).
Query	`ConsultarPosicionEnListaQuery` y `ConsultarReservasDelSocioQuery`	socioId	Handlers de lectura	`PosicionEnListaVm`, `ReservasDelSocioVm` (reserva, clase, estado)	US-16 y US-17.
Query	`ConsultarIndicadoresQuery`	rango de fechas	`ConsultarIndicadoresHandler`	`IndicadoresVm` (ocupación por clase y horario, no-shows por socio, tamaño promedio de lista por clase)	Indicadores del BRD (N-07, US-11, US-18).

Qué no se separa: `validarAcceso` es una consulta, pero usa el dominio (`ReglasDeElegibilidad`) y no una vista SQL. Duplicar BR-06 y BR-07 en una vista crearía dos fuentes de verdad, justo el origen de los accesos inconsistentes (N-01). Las operaciones administrativas son Commands simples y no necesitan modelo de lectura propio.

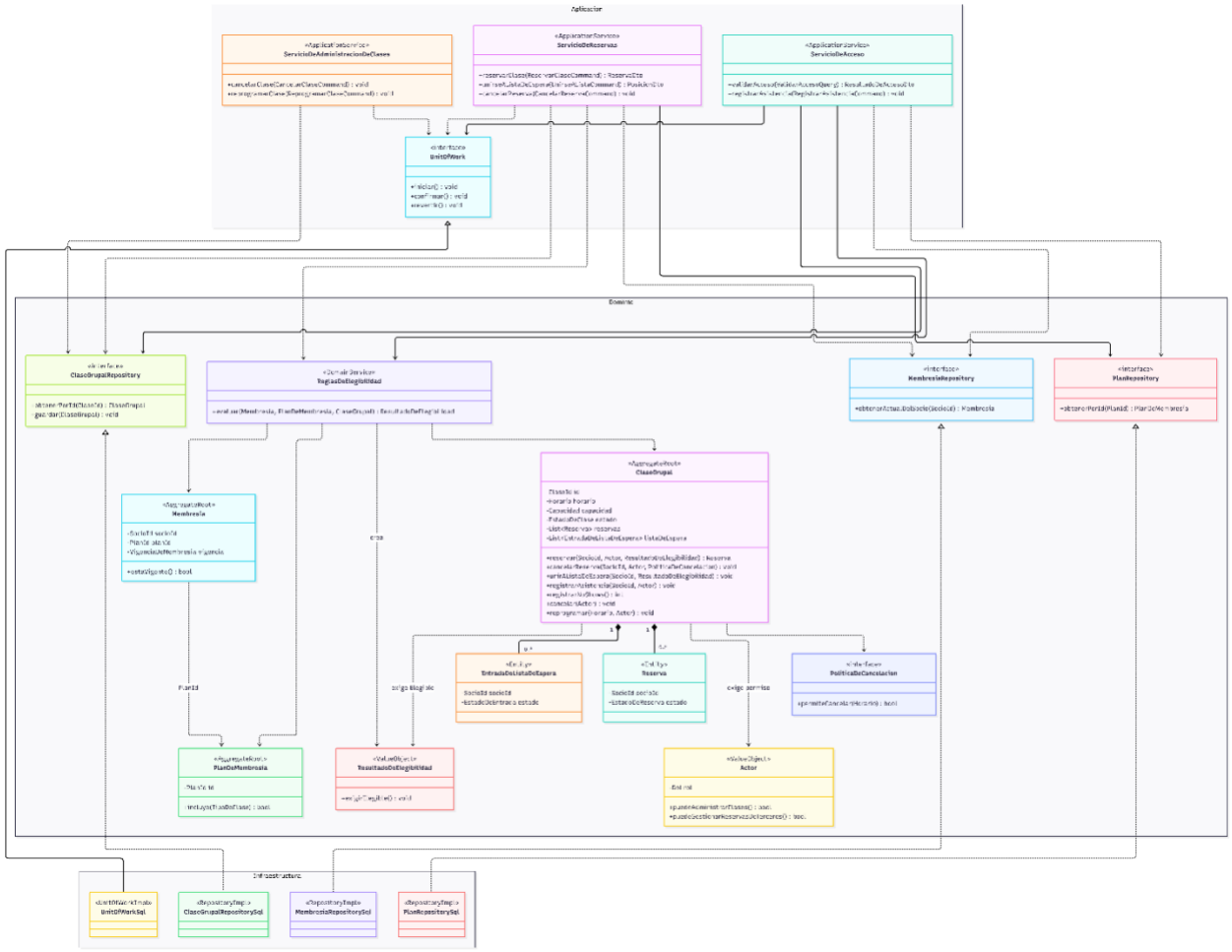
7. Revisión con SOLID (Etapa 7)

#	Principio	Evidencia	Impacto si no se corrige	Decisión o cambio en el diseño
1	S	Un servicio que evaluara vigencia, plan y cupo mezclaría tres razones de cambio.	Un cambio de BR-07 tocaría cuatro casos de uso.	Las reglas se ubican en `ReglasDeElegibilidad` y `ClaseGrupal`. `ServicioDeReservas` solo coordina.
2	S	Las acciones del socio (BR-10) y el cierre de clase (BR-08, BR-09) cambian por motivos distintos.	Una clase «dios» con cambios cruzados.	Se separa `ServicioDeCierreDeClase` de `ServicioDeReservas`.
3	O	BR-09 y BR-10 están sin definir y cambiarán.	Cadenas de `if` por cada nueva política.	Interfaces `PoliticaDeCancelacion` y `ConsecuenciaDeNoShow` con implementaciones provisionales (`PoliticaDeCancelacionProvisional`, `SinConsecuenciaDeNoShow`).
4	L	Derivar `Recepcion` de `Socio` heredaría permisos y rompería BR-03 y BR-04.	Sustituciones inseguras.	Los roles son un Value Object (`Rol`). Las políticas comparten un contrato puro (`permiteCancelar` sin efectos secundarios).
5	I	Una interfaz grande de notificación obligaría a implementar métodos irrelevantes.	Acoplamiento innecesario.	`NotificadorDeSocios` (oferta de lugar, cambio de clase) y `NotificadorDeAdministracion` (indisponibilidad). Repositories por Aggregate y puertos de lectura separados.
6	D	Un dominio que llame al reloj del sistema o a SQL no se puede probar.	BR-08 (fin del horario) no verificable.	Aplicación y dominio dependen de interfaces: Repository, `UnitOfWork`, `Reloj`, notificadores. La infraestructura los implementa.

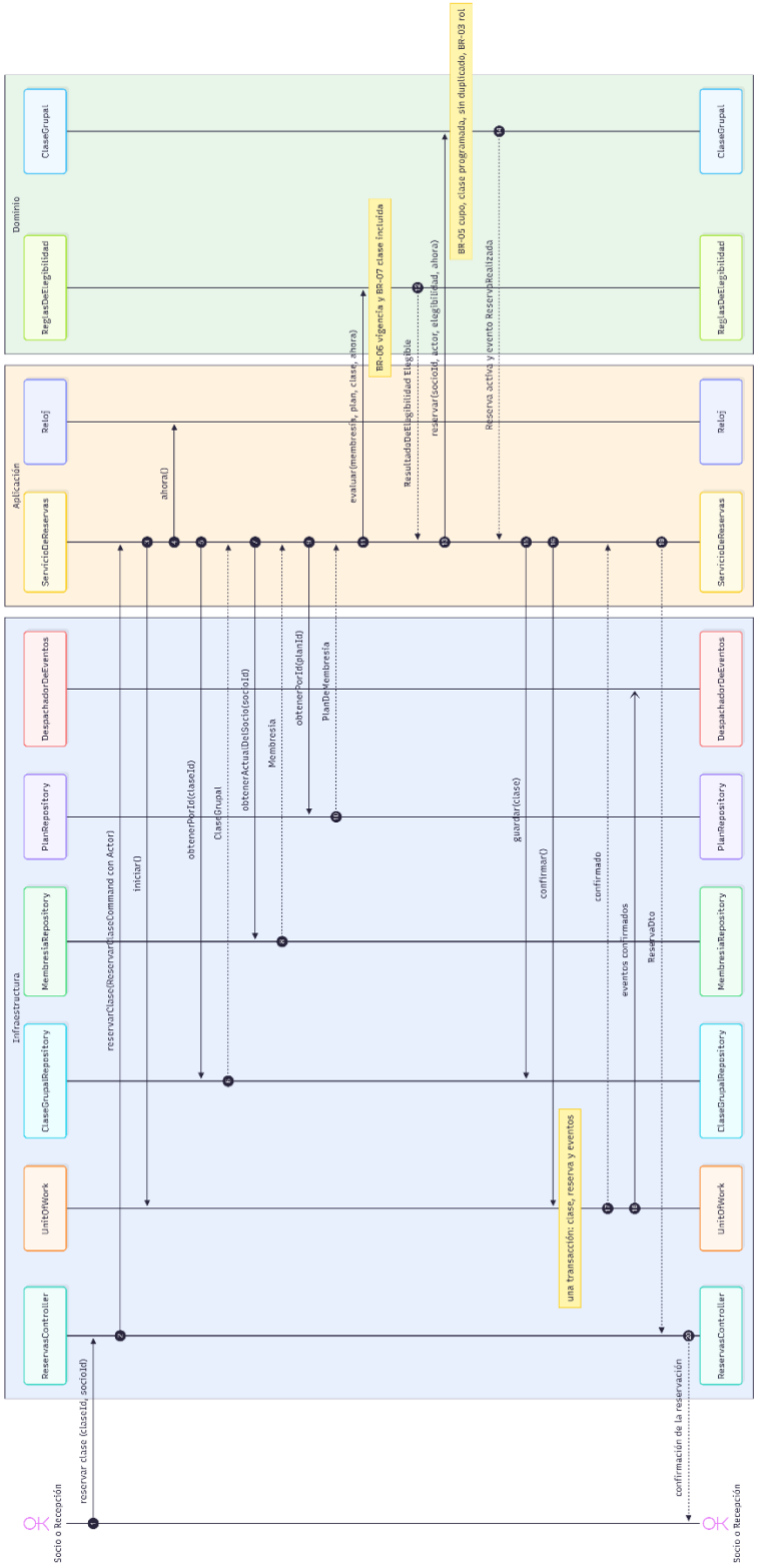
8. Representación de la arquitectura (Etapa 8)

8.1 Diagrama de clases (dominio, aplicación e infraestructura)

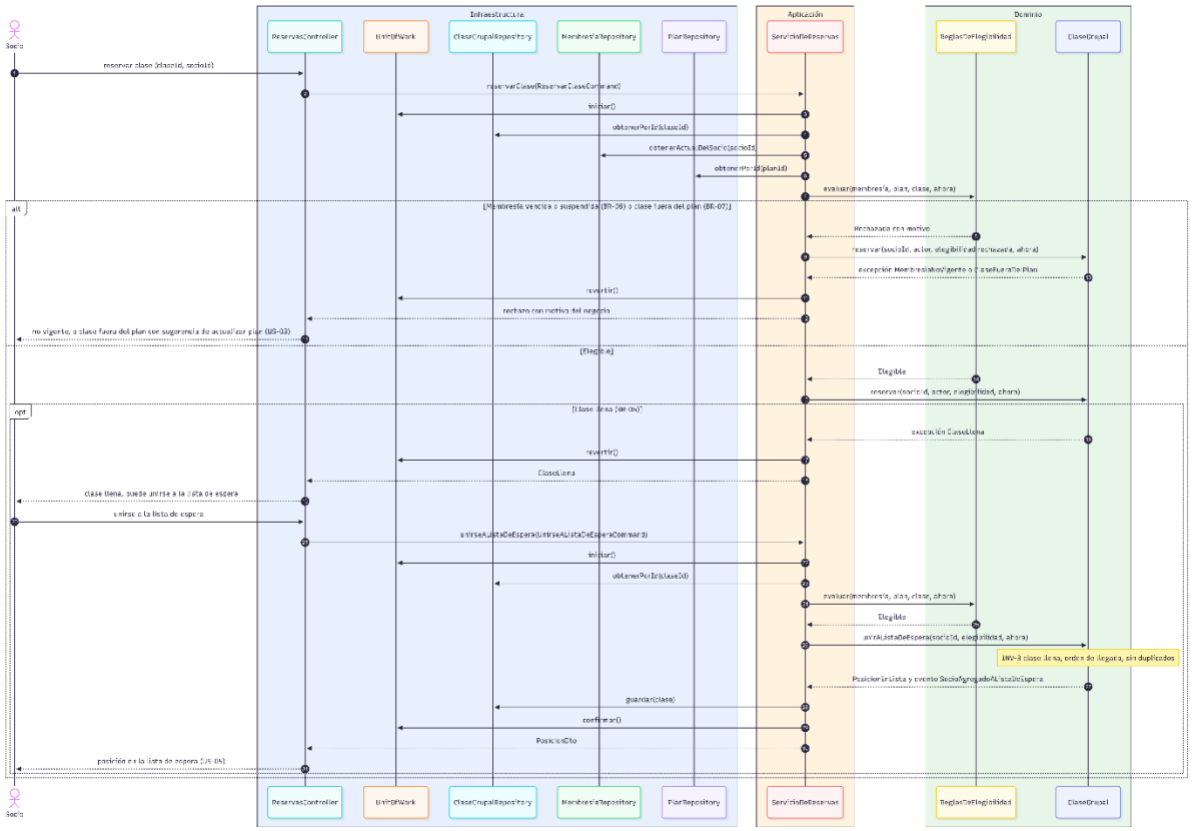
Separación de capas: el dominio no depende de nada de las otras dos. La aplicación depende del dominio y de puertos (interfaces) que ella misma declara. La infraestructura implementa Repositories y puertos y llama a la aplicación.



8.2 Secuencia 1: reservar una clase incluida en el plan (flujo normal, CU-01)



8.3 Secuencia 2: reserva rechazada y unión a la lista de espera (flujo alterno, CU-01 y CU-02)



Para cancelar una reserva (CU-03) la cadena es análoga: `cancelarReserva` llama a `ClaseGrupal.cancelarReserva`, que consulta `PoliticaDeCancelacion` (BR-10), libera el lugar y, si hay lista, ofrece el lugar al primero (`LugarOfrecidoASocio`). El `NotificadorDeSocios` se invoca después del commit, mediante el manejador de eventos.

9. Architecture Decision Records

ADR-001: Domain Model como patrón de lógica de negocio

- **Estado:** Aceptada.
- **Contexto:** reservar exige combinar membresía vigente (BR-06), clase incluida en el plan (BR-07) y cupo (BR-05) en el orden de los gateways del BPMN-1. Las mismas reglas se repiten en lista de espera, reserva por Recepción y acceso (US-04, US-05, US-08). Reserva y lista de espera tienen ciclos de vida con tiempo (US-06, US-10, BR-08). Las políticas BR-09 y BR-10 y el tiempo de confirmación están sin definir. Los indicadores (N-07) son lecturas tabulares.
- **Alternativas:** (1) Transaction Script: simple al inicio, pero duplica las reglas en cada caso de uso y facilita los accesos inconsistentes que N-01 quiere evitar. (2) Table Module: bueno para reportes, pero ningún módulo es dueño de las reglas que cruzan Membresía, Plan y Clase. (3) Domain Model para comandos con lectura directa para consultas.
- **Decisión:** alternativa 3. El dominio concentra las reglas de BR-01 a BR-08 y las políticas como estrategias. Las consultas e indicadores usan lectura directa (CQRS básico).
- **Consecuencias:** (+) reglas con dueño, vocabulario del glosario y pruebas unitarias de invariantes. (−) más clases, mapeo ORM y mayor curva de aprendizaje. (−) cuidado con concurrencia en `ClaseGrupal`. Trabajo posterior: validar SP-01 a SP-03 antes de la implementación definitiva.
- **Trazabilidad:** N-01 a N-05; BR-01 a BR-08; US-02, US-03, US-05, US-06, US-08, US-10, US-19; BPMN-1 y BPMN-2.

ADR-002: `ClaseGrupal` como Aggregate que contiene reservas y lista de espera

- **Estado:** Aceptada.
- **Contexto:** la capacidad máxima nunca debe superarse (BR-05, N-05). Cuando se libera un lugar, el primero de la lista debe recibirlo y el lugar queda apartado mientras confirma (US-06). Dos socios pueden pedir el último lugar al mismo tiempo.
- **Alternativas:** (a) `Reserva` y `ListaDeEspera` como Aggregates separados, con un servicio que cuente reservas: la invariante de cupo quedaría repartida y expuesta a condiciones de carrera. (b) `ClaseGrupal` como raíz que contiene `Reserva` y `EntradaDeListaDeEspera`. (c) Crear un registro de lugar por cada cupo: más tablas y no resuelve el orden de la lista.
- **Decisión:** alternativa b. Una clase, con capacidad acotada (BR-05), es un límite de consistencia pequeño. Se usa versión para concurrencia optimista. Historial, avisos y membresías son Aggregates aparte, referidos por identificador.

- **Consecuencias:** (+) INV-1 y INV-3 se garantizan en una sola transacción. (+) la oferta de lugar es atómica con la cancelación. (−) cada reserva o cancelación bloquea optimistamente toda la clase y puede requerir un reintento. (−) si FitZone tuviera clases con cientos de lugares, habría que revisar el límite. Trabajo posterior: pruebas de concurrencia del último lugar.
- **Trazabilidad:** BR-05, BR-07; N-03, N-05; US-02, US-05, US-06, US-07, US-13.

ADR-003: Eventos de dominio con Unit of Work y bandeja de salida

- **Estado:** Aceptada.
- **Contexto:** al ofrecer un lugar, cambiar una clase o registrar un no-show, otros componentes deben enterarse: notificar a socios (US-06, US-15), avisar al Administrador (US-14) y actualizar el historial (US-10, indicador de no-shows por socio). El canal de notificación aún no está definido (SP-07).
- **Alternativas:** (1) Llamar al notificador y al historial desde el servicio: acopla el servicio y mezcla efectos con la transacción. (2) Actualizar el historial en la misma transacción que la clase: modificar dos Aggregates en una transacción y bloquear más. (3) Eventos de dominio guardados junto al Aggregate en una bandeja de salida y despachados después del commit.
- **Decisión:** alternativa 3. `UnitOfWork.confirmar()` persiste el Aggregate y sus eventos juntos. Un despachador entrega cada evento a manejadores que usan sus propias transacciones.
- **Consecuencias:** (+) notificaciones confiables sin atar la reserva al canal; (+) cambiar de canal no toca el dominio. (−) el historial es consistente en forma eventual. (−) los manejadores deben ser idempotentes porque un evento puede entregarse más de una vez. Trabajo posterior: definir reintentos y métricas de eventos pendientes.
- **Trazabilidad:** N-02, N-03, N-06, N-07; BR-08, BR-09; US-06, US-10, US-14, US-15.

ADR-004: Políticas aún no definidas como estrategias configurables

- **Estado:** Propuesta (requiere validación de FitZone).
- **Contexto:** el BRD declara que no existe política uniforme de cancelación (BR-10), ni consecuencia formal del no-show (BR-09), ni tiempo límite para confirmar un lugar de la lista de espera. También indica que las políticas deben establecerse antes de la implementación definitiva.
- **Alternativas:** (1) Escribir valores fijos en el código: inventa reglas de negocio. (2) Esperar a que se definan antes de diseñar: bloquea el avance. (3) Estrategias en el dominio (`PoliticaDeCancelacion`, `ConsecuenciaDeNoShow`) y un Value Object configurable (`VentanaDeConfirmacion`), con implementaciones provisionales marcadas.
- **Decisión:** alternativa 3. `PoliticaDeCancelacionProvisional` permite cancelar antes del inicio de la clase (SP-01). `SinConsecuenciaDeNoShow` solo registra el no-show (BR-

09). `VentanaDeConfirmacion` se inyecta por configuración, sin valor por defecto (SP-02).

- **Consecuencias:** (+) el negocio podrá fijar sus políticas sin reescribir casos de uso (principio abierto/cerrado). (−) riesgo de que el valor provisional se tome por definitivo: debe validarse con FitZone y quedar visible en pruebas y README. Trabajo posterior: reemplazar las implementaciones cuando exista la política.
- **Trazabilidad:** BR-09, BR-10; Restricciones y Supuestos del BRD; SP-01, SP-02, SP-03; US-06, US-07, US-10.

10. Matriz de trazabilidad

Necesidad	Regla	Historia	BPMN	Caso de uso	Componentes	ADR
N-01 Control de acceso	BR-06	US-08, US-19	BPMN-2: acceso válido	CU-04	`ServicioDeAcceso`, `ReglasDeElegibilidad`, `Membresia`	001
N-02 Reducir no-shows	BR-08, BR-09	US-10, US-19	BPMN-2: se presentó	CU-05	`ServicioDeCierreDeClase`, `ClaseGrupal.registrarNoShows`, `HistorialDeAsistencia`, `ConsecuenciaDeNoShow`	003, 004
N-03 Lista de espera	BR-05, BR-10	US-05, US-06, US-07, US-16	BPMN-1: lista de espera	CU-02, CU-03	`EntradaDeListaDeEspera`, `ClaseGrupal`, `ServicioDeReservas`, `NotificadorDeSocios`	002, 003, 004
N-04 Clase según plan	BR-07	US-02, US-03	BPMN-1: incluida en el plan	CU-01	`ReglasDeElegibilidad`, `PlanDeMembresia`	001
N-05 Control de capacidad	BR-05	US-02, US-13	BPMN-1: hay cupo	CU-01, CU-02	`Capacidad`, `ClaseGrupal` (INV-1)	002
N-06 Administración de clases	BR-01, BR-02	US-12, US-14, US-15	Evento instructor no puede	Servicios secundarios	`ServicioDeAdministracionDeClases`, `ServicioDeIndisponibilidad`,	003

Necesidad	Regla	Historia	BPMN	Caso de uso	Componentes	ADR
					`AvisoDeIndisponibilidad`, `Actor`	
N-07 Indicadores	(indicadores del BRD)	US-11, US-17, US-18	—	Consultas	Handlers de consulta, `LecturaDeIndicadores`	001
Reservas por Recepción	BR-03, BR-04	US-04	BPMN-1: Recepción / Sistema	CU-01, CU-03	`Actor`, `Rol`, `ClaseGrupal`	001

11. Validación y cierre (Etapa 9)

Validación	Resultado del diseño
Trazabilidad	Cada componente de la matriz responde a una necesidad, regla, historia o caso de uso.
Lenguaje	Los términos del glosario se usan en BRD, UML, ADR y firmas. Los siete términos nuevos están propuestos.
Responsabilidad	Las reglas viven en `ClaseGrupal`, `ReglasDeElegibilidad`, `Membresia`, `PlanDeMembresia`, `Actor` y las políticas. Los servicios solo coordinan.
Consistencia	Las secuencias usan los componentes del diagrama de clases.
Persistencia	El dominio solo conoce interfaces de Repository. El SQL y el ORM están en infraestructura.
Transacción	Una por Command, con `confirmar()` y `revertir()` definidos y un solo Aggregate modificado.
IA	Se registró una bitácora con 8 iteraciones (prompt, respuesta útil, error o supuesto y decisión del equipo). Los aportes de la IA se contrastaron con el BRD, las historias de usuario y el BPMN. Se detectaron y corrigieron errores: una redacción incorrecta en la tabla SOLID y el desalineamiento entre el diagrama de clases simplificado y otras secciones.

Bitácora de IA

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
1	Preparar el caso y elegir los casos de uso	Se pegan fragmentos del BRD (alcance, BR-01 a BR-10), las historias US-01 a US-19 y los dos BPMN. Pide: "Propón de 3 a 5 casos de uso del proceso principal, con actor, reglas y resultado. Declara tus supuestos y no inventes reglas."	Cinco casos de uso (reservar, unirse a la lista de espera, cancelar y ofrecer lugar, validar acceso y registrar asistencia, registrar no-show), cada uno ligado a historias y reglas	La IA propone incluir US-09 (servicio adicional), que depende de pagos, fuera del alcance del BRD	Descartar US-09 y registrarlo como corrección de alcance. Verificar a mano que cada caso aparezca en historias o BPMN
2	Revisar la Unidad 2 antes de diseñar	"Busca contradicciones entre BRD, historias y BPMN. No propongas arquitectura todavía."	Lista de inconsistencias: etiqueta BR-03 mal asignada en el BPMN, no-show que libera el lugar al terminar la clase, acceso con reserva frente al plan básico sin reserva	La IA marca BR-06 como ambigua ("no debería") y asume que es obligatoria	El equipo confirma cada hallazgo contra los documentos. Corrige el BPMN y registra los cambios en la trazabilidad. Lleva a FitZone las preguntas sin respuesta
3	Comparar patrones de lógica de negocio	"Con base en [reglas, cambios y casos de uso], compara Transaction Script, Table Module y Domain Model. Declara	Tabla comparativa con criterios del caso: reglas combinadas, políticas cambiantes, ciclos de vida, permisos	La IA argumenta que Domain Model es "más escalable", sin evidencia del caso	Rechazar el argumento. Reescribir la justificación con tres evidencias de la Unidad 2: BR-05/06/07 en cascada, ciclos de vida de US-06 y US-

		supuestos y no elijas hasta mostrar consecuencias.”			10, y políticas pendientes BR-09/10. Registrar el costo aceptado
4	Proponer el mapa de dominio	“Revisa este glosario y estas reglas. Propón entidades, Value Objects, Aggregates y eventos, con evidencia y dudas para validación humana.”	Candidatos a Aggregates, entre ellos ClaseGrupal, Membresia e HistorialDeAsistencia, con invariantes y eventos	La IA convierte Reserva, ListaDeEspera y Socio en Aggregates independientes, casi uno por tabla	Discutir en equipo la consistencia del cupo. Fusionar reserva y lista dentro de ClaseGrupal. Dejar Socio solo como identificador. Registrar la decisión en un ADR
5	Diseñar la Service Layer	“A partir de estos casos de uso, propone firmas de Application Services. Separa coordinación de reglas y explica cualquier responsabilidad dudosa.”	Servicios por caso de uso con colaboradores y reglas delegadas	La IA coloca la validación de vigencia y plan dentro de reservarClase	Mover esas validaciones a ReglasDeElegibilidad y a ClaseGrupal. Verificar que ningún servicio contenga reglas de negocio
6	Revisar con SOLID	“Analiza este diseño y señala posibles violaciones SOLID. No reescribas todo: da evidencia, riesgo y alternativas.”	Hallazgos sobre servicios con dos razones de cambio, políticas sin abstracción y dependencias directas de infraestructura	La IA sugiere herencia entre roles (Recepcion hereda de Socio)	Rechazar la herencia por romper BR-03 y BR-04. Modelar el rol como Value Object. Separar el servicio de cierre de clase
7	Producir diagramas coherentes	“Convierte esta arquitectura validada en Mermaid. No	Borradores de diagrama de clases y dos de secuencia	Los participantes de la secuencia no coinciden	El equipo concilia nombres entre diagramas. Recorta el diagrama a lo

		agregues clases, reglas ni relaciones que no aparezcan en la descripción.”		con las clases. Quedan clases ajenas a BR-01 a BR-10	estrictamente necesario. Verifica que cada flujo funcione sobre las clases mostradas
8	Registrar las decisiones en ADR y validar la trazabilidad	“Redacta ADR con título, estado, contexto, alternativas, decisión, consecuencias y trazabilidad, usando solo las decisiones que ya tomamos.”	Borradores de ADR y una matriz de trazabilidad	La IA da por definidos valores de políticas, como el plazo de cancelación	Marcar esos valores como supuestos pendientes. Ajustar los ADR. Verificar que los IDs de necesidades, reglas e historias existan

Conclusión

El análisis de la Unidad 2 mostró que el problema central de FitZone no es de captura de datos, sino de reglas y estados. Reservar exige combinar membresía vigente, clase incluida en el plan y cupo disponible. La lista de espera y el no-show tienen ciclos de vida con tiempo. Por eso se eligió el Domain Model como patrón principal: ubica cada regla en el objeto que la posee, con el lenguaje del glosario, y protege invariantes como la capacidad máxima. Las consultas e indicadores se resuelven con lectura directa, para no sobrecargar el dominio. La Service Layer solo coordina, los Repositories y el Unit of Work aíslan la persistencia, y los eventos de dominio desacoplan las notificaciones y el historial.

Como consecuencia aceptada, la arquitectura tiene mayor costo inicial de modelado y exige control de concurrencia sobre la clase, sobre todo con el último lugar disponible. A cambio, el sistema queda preparado para reducir los no-shows y los lugares vacíos, automatizar la lista de espera, evitar accesos inconsistentes y dar a la administración información confiable sobre la demanda.